VALIDATION PLAN
OPERATIONAL QUALIFICATION
CURVE PROPERTIES MODULE

JR Validated Environment — Curve Properties Module v1.0

Field	Value
Document Number	JR-VP-CURVE-001
Title	Validation Plan — Operational Qualification, Curve Properties Module
System	JR Validated Environment — Curve Properties Module
Module Version	1.0
Document Version	1.0
Status	Approved
Effective Date	2026-03-26
Author	Joep Rous
Reviewer	[Name]
Approver	[Name]

Version History

Version	Date	Author	Description
1.0	2026-03-26	Joep Rous	Initial release. Covers jrc_curve_properties (Curve Properties Module v1.0).

Approval Signatures

Role	Name	Signature	Date
Author	Joep Rous		2026-03-26
Reviewer	[Name]		
Approver (QA)	[Name]		

1. Purpose

This document defines the Validation Plan for the Operational Qualification (OQ) of the Curve Properties module included in version 1.0 of the JR Validated Environment. It specifies the user requirements, test cases, acceptance criteria, and traceability between requirements and tests needed to demonstrate that jrc_curve_properties performs correctly for its intended use.

Execution of the tests defined in this plan, together with satisfactory results and documented evidence, constitutes the OQ for the Curve Properties module.

2. Scope

2.1 In Scope

- The Python script repos/curve/Python/jrc_curve_properties.py
- Config-file validation (pre-flight detection of errors before data loading)
- Data loading from a two-column CSV file specified in the config
- Phase extraction — segmenting the curve into named phases by row-proximity search
- Global and per-phase statistics: max_y, min_y, max_x, min_x, area under curve (AUC)
- Hysteresis calculation between two named phases
- Slope calculations: overall linear regression, secant, and instantaneous at a given X
- Query operations: Y at X, X at Y, and Y at relative X
- Transform operations: Y scale and Y offset
- Transition detection: inflection points and yield points
- Output verification: results text file, PDF plot, and debug second-derivative CSV
- Correct rejection of invalid inputs with informative error messages

2.2 Out of Scope

- Infrastructure scripts in bin/ and admin/ (covered by IQ; see JR-VP-001)
- Performance qualification (PQ) with real production measurement data
- User acceptance testing (UAT) with application-specific datasets
- Validation of Python interpreter and third-party packages (numpy, scipy, matplotlib) — package integrity is confirmed at installation via SHA256 (JR-IQ-001)
- Statistical validation of the Savitzky-Golay smoothing algorithm itself — the original reference is cited in Section 3
- Validation of PDF rendering by matplotlib

3. Regulatory Basis and References

3.1 Regulations

Reference	Title
21 CFR Part 820, §820.70(i)	Quality System Regulation — Automated Data Processing
21 CFR Part 820, §820.75	Process Validation
21 CFR Part 11	Electronic Records; Electronic Signatures
ISO 13485:2016, §7.5.6	Validation of processes for production and service provision

3.2 FDA Guidance

Reference	Title	Date
FDA GPSV	General Principles of Software Validation; Final Guidance for Industry and FDA Staff	January 2002
FDA CSA	Computer Software Assurance for Production and Quality System Software; Draft Guidance	September 2022

3.3 Methods and Standards

Reference	Title
Savitzky A, Golay MJE (1964)	Smoothing and Differentiation of Data by Simplified Least Squares Procedures. Analytical Chemistry, 36(8), 1627–1639. Original publication of the Savitzky-Golay smoothing filter used for derivative calculations.
ISO 13485:2016	Medical devices — Quality management systems — Requirements for regulatory purposes.
NumPy / SciPy documentation	scipy.signal.savgol_filter — implementation of the Savitzky-Golay filter. numpy.trapezoid — composite trapezoidal numerical integration for AUC.

3.4 Internal Documents

Document Number	Title
JR-VP-001	Validation Plan — Installation Qualification, JR Validated Environment v1.0.0
JR-IQ-001	IQ Execution Evidence (docs/IQ_validation_20260311_205146.txt)
JR-VP-002	Validation Plan — OQ, Community Script Suite v1.1.0

4. Definitions and Abbreviations

Term	Definition
Phase	A named contiguous segment of the XY curve, defined by x_start and x_end in a [phase.NAME] config section. Phase boundaries are located by row-proximity search (argmin of |x − x_boundary|) in time order.
AUC	Area under curve, computed via the composite trapezoidal rule (numpy.trapezoid) on raw (unsmoothed) x and y data.
Hysteresis	The enclosed area between the loading and unloading arms of a curve, computed as the integral of |y_loading(x) − y_unloading(x)| over the overlapping x range using linear interpolation and 500 equally spaced points.
Savitzky-Golay filter	A polynomial-fitting smoothing filter (Savitzky & Golay, 1964) used to compute smooth derivatives. For inflection detection, the filter is applied with deriv=2 (direct second-derivative computation) to avoid double-gradient boundary amplification.
Inflection point	A point on the curve where the second derivative changes sign, indicating a change in the direction of curvature. Detected by smoothed d^2y/dx^2 sign change with a configurable minimum separation (min_gap).
Yield point	The first point after the maximum-slope region where the instantaneous slope drops to a specified fraction (yield.slope) of the maximum slope. Used to identify the elastic-to-plastic transition on force-displacement curves.
Secant slope	The slope of the straight line connecting two specified x values on the curve: (y(x2) − y(x1)) / (x2 − x1), computed using linear interpolation.
Config file	An INI-format text file (.cfg) that is the sole CLI argument to jrc_curve_properties.py. All analysis options are specified as key=value pairs in sections such as [data], [global], [phase.NAME], [slope], etc.
Dot notation	The modifier key convention used in the config: feature.modifier (e.g. secant.x1, yield_1.slope, y_at_x_1.show). Dots unambiguously separate a feature name from its modifier, replacing the earlier underscore convention.
IQ	Installation Qualification

OQ	Operational Qualification
PQ	Performance Qualification
UR	User Requirement
TC	Test Case
jrrun	The validated bash entry-point script that sets the controlled Python virtual environment before executing any script
cfg	Configuration file in INI format; the sole input argument to jrc_curve_properties.py
AUC	Area Under Curve
CSV	Comma-separated values input file
PDF	Portable Document Format; optional plot output

5. User Requirements

The following user requirements define the expected behaviour of jrc_curve_properties. Each requirement is verifiable by the test cases in Section 7.

UR ID	Title	Requirement
UR-CURVE-001	Config file input	The script shall accept a single config file path (.cfg, INI format) as its sole CLI argument. All analysis options shall be specified in the config file.
UR-CURVE-002	Data loading from CSV	The script shall load x and y arrays from the CSV file specified in the [data] section, using x_col and y_col to identify the measurement columns.
UR-CURVE-003	Phase extraction	The script shall extract named curve phases from [phase.NAME] config sections. Phase boundaries (x_start, x_end) shall be located by row-proximity search (argmin of |x − x_boundary|) in time order, optionally restricted by search = ascending | descending.
UR-CURVE-004	Global statistics	When enabled in [global], the script shall compute max_y, min_y, max_x, and min_x over the full dataset and report the position of each extremum.
UR-CURVE-005	Area under curve (AUC)	When auc=yes is set in [global] or a [phase.NAME] section, the script shall compute the area under the curve using the composite trapezoidal rule (numpy.trapezoid) on raw, unsmoothed data.
UR-CURVE-006	Hysteresis	When hysteresis=yes is set in [global] with loading and unloading phase names specified, the script shall compute the enclosed area between the two arms using linear interpolation over their overlapping x range.
UR-CURVE-007	Per-phase statistics	When max_y, min_y, max_x, min_x, or auc are declared in a [phase.NAME] section, the script shall compute those statistics for that phase only and label the results with the phase name (e.g. 'max Y [loading]').
UR-CURVE-008	Slope calculations	The script shall support three slope modes: overall linear regression (with R^2), secant slope between two specified x values, and instantaneous slope at a given x using smoothed numerical derivatives.
UR-CURVE-009	Query operations	The script shall support three query types: Y at X (linear interpolation), X at Y (crossing detection with first/last/all modes), and Y at relative X (x_ref × (1 + frac) with signed fractional offset).
UR-CURVE-010	Transform operations	When [transform] is specified, y_scale shall multiply all Y values (applied first) and y_offset_x shall subtract Y at a reference x from all values. Transform parameters shall be reported in the results output.
UR-CURVE-011	Inflection points	When inflections or inflections_N=yes is set in [transitions], the script shall detect inflection points as sign changes of the smoothed second derivative (savgol_filter deriv=2) with configurable minimum gap between detections.
UR-CURVE-012	Yield point	When yield.slope or yield_N.slope is set in [transitions], the script shall find the first point after the maximum-slope region where the instantaneous slope drops to the specified fraction of the maximum

		slope.
UR-CURVE-013	Results text file	The script shall write a formatted text file containing all computed results in a labelled, aligned table. The output path shall be configurable via [output] results_file; default is <cfg_stem>_results.txt.
UR-CURVE-014	Plot output	When plot=yes is set in [output], the script shall generate a PDF plot of the curve data with all enabled annotations (phases, markers, tangent lines, regression lines). The output path shall be configurable via [output] plot_file.
UR-CURVE-015	Debug second-derivative output	When [debug] d2y=yes is set with d2y.phase specified, the script shall write a CSV file named <cfg_stem>_debug_d2y_<phase>.csv containing columns x, y_raw, y_smooth, d2y, and trimmed for the specified phase.
UR-CURVE-016	Config pre-validation	The script shall validate the config file before any data loading and collect all errors and warnings in a single pass. Errors shall include: unknown sections, unknown keys, non-numeric values, missing required keys, and d2y without d2y.phase. The script shall exit non-zero if any errors are found, reporting all of them.
UR-CURVE-017	Error handling — non-zero exit	The script shall exit non-zero with an informative message for any error condition, including missing config file, unreadable data file, and config validation failures.
UR-CURVE-018	Help mode	When invoked with no arguments or the --help flag, the script shall print usage information and exit 0. This provides onboarding guidance without requiring a config file.

6. Test Environment

Component	Specification
Operating System	macOS 15.x (Apple Silicon) or Windows 10/11 (Git Bash)
Python Version	As installed per JR-IQ-001 (Python 3.11.x or later)
Test Framework	pytest 8.x
Test runner	repos/curve/admin_curve_oq
Test directory	repos/curve/oq/
Test data	repos/curve/oq/data/ (3 CSV fixtures, 15 config files)
OQ venv	~/.venvs/<PROJECT_ID>_oq/ (shared with other module OQ suites)
Entry point	bin/jrrun (integrity-checked execution)
Script venv	~/.venvs/<PROJECT_ID>/ (scipy, numpy, matplotlib)

7. Test Cases

All test cases are implemented in repos/curve/oq/test_curve_properties.py. Test data files (CSV datasets and config files) reside in repos/curve/oq/data/. All commands are run via bin/jrrun from the repos/curve/oq/data/ directory.

7.1 Config Validation

TC-CURVE-V-001

Tests: UR-CURVE-018
Command: `jrrun jrc_curve_properties.py  (no arguments)`
Rationale: No config path supplied; script enters help mode.
Pass criterion: exit 0; 'USAGE' or 'Usage' in output

TC-CURVE-V-002

Tests: UR-CURVE-017
Command: `jrrun jrc_curve_properties.py nonexistent_config_xyz.cfg`
Rationale: Config file does not exist on disk.

Pass criterion: exit non-zero

TC-CURVE-V-003

Tests: UR-CURVE-017
Command: `jrrun jrc_curve_properties.py oq/data/err_no_datafile.cfg`
Rationale: Config valid; data file referenced does not exist.
Pass criterion: exit non-zero

TC-CURVE-V-004

Tests: UR-CURVE-016
Command: `jrrun jrc_curve_properties.py oq/data/err_unknown_section.cfg`
Rationale: Config contains [badstuff] — not a recognised section.
Pass criterion: exit non-zero; 'Unknown section' in output

TC-CURVE-V-005

Tests: UR-CURVE-016
Command: `jrrun jrc_curve_properties.py oq/data/err_unknown_key.cfg`
Rationale: Config contains 'max_yy' in [global] — typo key.
Pass criterion: exit non-zero; 'Unknown key' in output

TC-CURVE-V-006

Tests: UR-CURVE-016
Command: `jrrun jrc_curve_properties.py oq/data/err_nonnumeric.cfg`
Rationale: Config contains y_scale = abc in [transform].
Pass criterion: exit non-zero

TC-CURVE-V-007

Tests: UR-CURVE-016
Command: `jrrun jrc_curve_properties.py oq/data/err_d2y_no_phase.cfg`
Rationale: [debug] d2y=yes present but d2y.phase is absent.
Pass criterion: exit non-zero

7.2 Global Statistics

TC-CURVE-G-001

Tests: UR-CURVE-004
Command: `jrrun jrc_curve_properties.py oq/data/test_global.cfg`
Rationale: global max_y=yes on linear data (max = 40 at x=20).
Pass criterion: exit 0; 'max Y' in output

TC-CURVE-G-002

Tests: UR-CURVE-004
Command: `jrrun jrc_curve_properties.py oq/data/test_global.cfg`
Rationale: global min_y=yes on linear data (min = 0 at x=0).
Pass criterion: exit 0; 'min Y' in output

TC-CURVE-G-003

Tests: UR-CURVE-004
Command: `jrrun jrc_curve_properties.py oq/data/test_global.cfg`
Rationale: global max_x=yes and min_x=yes on linear data.
Pass criterion: exit 0; 'max X' and 'min X' in output

TC-CURVE-G-004

Tests: UR-CURVE-005

Command: `jrrun jrc_curve_properties.py oq/data/test_global.cfg`
Rationale: global auc=yes on linear data y=2x, x=0..20. Expected AUC = 400 (trapezoid rule, exact on linear data).
Pass criterion: exit 0; 'AUC' and '400' in output

TC-CURVE-G-005

Tests: UR-CURVE-006
Command: `jrrun jrc_curve_properties.py oq/data/test_hysteresis.cfg`
Rationale: hysteresis=yes with loading (y=10x) and unloading (y=9x) phases on triangle data.
Pass criterion: exit 0; 'Hysteresis' in output

7.3 Per-Phase Statistics

TC-CURVE-P-001

Tests: UR-CURVE-007
Command: `jrrun jrc_curve_properties.py oq/data/test_phase.cfg`
Rationale: [phase.loading] max_y=yes; loading arm x=0..10, y=0..100.
Pass criterion: exit 0; 'max Y [loading]' in output

TC-CURVE-P-002

Tests: UR-CURVE-005, UR-CURVE-007
Command: `jrrun jrc_curve_properties.py oq/data/test_phase.cfg`
Rationale: [phase.loading] auc=yes; loading AUC = trapezoid(y=10x, 0..10) = 500.
Pass criterion: exit 0; 'AUC [loading]' in output

TC-CURVE-P-003

Tests: UR-CURVE-003, UR-CURVE-007
Command: `jrrun jrc_curve_properties.py oq/data/test_phase.cfg`
Rationale: Two phases declared: [phase.loading] and [phase.unloading].
Pass criterion: exit 0; 'loading' and 'unloading' both present in output

7.4 Slope Calculations

TC-CURVE-S-001

Tests: UR-CURVE-008
Command: `jrrun jrc_curve_properties.py oq/data/test_slope.cfg`
Rationale: overall=yes; linear regression on y=2x data. Expected slope ≈2.
Pass criterion: exit 0; 'slope overall' in output

TC-CURVE-S-002

Tests: UR-CURVE-008
Command: `jrrun jrc_curve_properties.py oq/data/test_slope.cfg`
Rationale: secant=yes with secant.x1=2, secant.x2=8 on linear data.
Pass criterion: exit 0; 'slope secant' in output

TC-CURVE-S-003

Tests: UR-CURVE-008
Command: `jrrun jrc_curve_properties.py oq/data/test_slope.cfg`
Rationale: at_x_1=10; instantaneous slope at x=10 on linear data.
Pass criterion: exit 0; 'slope at x=' in output

7.5 Query Operations

TC-CURVE-Q-001

Tests: UR-CURVE-009

Command: `jrrun jrc_curve_properties.py oq/data/test_query.cfg`
Rationale: y_at_x_1=5; Y at x=5 on linear data y=2x. Expected Y=10.
Pass criterion: exit 0; 'Y at x=' in output

TC-CURVE-Q-002

Tests: UR-CURVE-009
Command: `jrrun jrc_curve_properties.py oq/data/test_query.cfg`
Rationale: x_at_y_1=10; X where Y=10 on linear data. Expected X=5.
Pass criterion: exit 0; 'X at y=' in output

TC-CURVE-Q-003

Tests: UR-CURVE-009
Command: `jrrun jrc_curve_properties.py oq/data/test_query.cfg`
Rationale: y_at_rel_x_1=10 with y_at_rel_x_1.frac=0.1; queries Y at x=10×1.1=11.
Pass criterion: exit 0; 'Y at' and '%' in output

7.6 Transform Operations

TC-CURVE-T-001

Tests: UR-CURVE-010
Command: `jrrun jrc_curve_properties.py oq/data/test_transform.cfg`
Rationale: y_scale=0.5 in [transform]; all Y values multiplied by 0.5.
Pass criterion: exit 0; 'Y scale' in output

TC-CURVE-T-002

Tests: UR-CURVE-010
Command: `jrrun jrc_curve_properties.py oq/data/test_transform.cfg`
Rationale: y_offset_x=5 in [transform]; Y at x=5 subtracted from all Y.
Pass criterion: exit 0; 'Y offset' in output

7.7 Transitions

TC-CURVE-I-001

Tests: UR-CURVE-011
Command: `jrrun jrc_curve_properties.py oq/data/test_transitions.cfg`
Rationale: inflections_1=yes on sine data (y=sin(x), x=0..2π). Expected inflection at x≈π (sign change of d^2y/dx^2).
Pass criterion: exit 0; 'inflection' in output

TC-CURVE-I-002

Tests: UR-CURVE-012
Command: `jrrun jrc_curve_properties.py oq/data/test_transitions.cfg`
Rationale: yield_1.slope=0.5 on sine data. Max slope≈1.0 at x=0; yield expected at x≈π/3 where cos(x)=0.5.
Pass criterion: exit 0; 'yield point' in output

7.8 Output Files

TC-CURVE-O-001

Tests: UR-CURVE-013
Command: `jrrun jrc_curve_properties.py oq/data/test_plot.cfg`
Rationale: results_file=oq_test_results.txt in [output]; verifies file creation.
Pass criterion: exit 0; oq_test_results.txt exists in oq/data/

TC-CURVE-O-002

Tests: UR-CURVE-014

Command: `jrrun jrc_curve_properties.py oq/data/test_plot.cfg`
Rationale: plot=yes and plot_file=oq_test_plot.pdf in [output].
Pass criterion: exit 0; oq_test_plot.pdf exists in oq/data/

TC-CURVE-O-003

Tests: UR-CURVE-015
Command: `jrrun jrc_curve_properties.py oq/data/test_debug.cfg`
Rationale: [debug] d2y=yes with d2y.phase=full on sine data.
Pass criterion: exit 0; test_debug_debug_d2y_full.csv exists in oq/data/

8. Requirements Traceability Matrix

Each user requirement is covered by at least one test case.

UR ID	Title	Test Cases
UR-CURVE-001	Config file input	TC-CURVE-V-001..007, TC-CURVE-G-001..005, TC-CURVE-P-001..003, TC-CURVE-S-001..003, TC-CURVE-Q-001..003, TC-CURVE-T-001..002, TC-CURVE-I-001..002, TC-CURVE-O-001..003 (all use a config file)
UR-CURVE-002	Data loading from CSV	TC-CURVE-G-001..005, TC-CURVE-P-001..003 (results verified against known data values)
UR-CURVE-003	Phase extraction	TC-CURVE-P-003 (two phases extracted in one run)
UR-CURVE-004	Global statistics	TC-CURVE-G-001, TC-CURVE-G-002, TC-CURVE-G-003
UR-CURVE-005	AUC (trapezoid rule)	TC-CURVE-G-004, TC-CURVE-P-002
UR-CURVE-006	Hysteresis	TC-CURVE-G-005
UR-CURVE-007	Per-phase statistics	TC-CURVE-P-001, TC-CURVE-P-002, TC-CURVE-P-003
UR-CURVE-008	Slope calculations	TC-CURVE-S-001, TC-CURVE-S-002, TC-CURVE-S-003
UR-CURVE-009	Query operations	TC-CURVE-Q-001, TC-CURVE-Q-002, TC-CURVE-Q-003
UR-CURVE-010	Transform operations	TC-CURVE-T-001, TC-CURVE-T-002
UR-CURVE-011	Inflection points	TC-CURVE-I-001
UR-CURVE-012	Yield point	TC-CURVE-I-002
UR-CURVE-013	Results text file	TC-CURVE-O-001
UR-CURVE-014	Plot PDF output	TC-CURVE-O-002
UR-CURVE-015	Debug d2y CSV output	TC-CURVE-O-003
UR-CURVE-016	Config pre-validation	TC-CURVE-V-004, TC-CURVE-V-005, TC-CURVE-V-006, TC-CURVE-V-007
UR-CURVE-017	Error handling — non-zero exit	TC-CURVE-V-002, TC-CURVE-V-003
UR-CURVE-018	Help mode	TC-CURVE-V-001

9. Acceptance Criteria

The Curve Properties module OQ is considered PASS when all of the following conditions are satisfied:

- All 28 automated test cases execute with the result PASSED.
- Zero test cases result in FAILED or ERROR.
- The OQ evidence file is generated and preserved at ~/.jrscript/<PROJECT_ID>/validation/curve_oq_execution_<timestamp>.txt.
- All test cases are executed using the validated jrrun entry-point.
- No open deviations remain unresolved at the time of approval.

Any test failure shall be investigated, documented as a deviation, resolved, and the affected test(s) re-executed. The OQ may not be closed until all deviations are resolved.